Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using pytorch.

2 Learning Outcomes

Students will be able to:

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network consists of:

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is:

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n) \tag{1}$$

where

- X: Input Image
- K: Kernel (Filter)
- Y: Feature Map

The output feature map size is:

$$\frac{N - F + 2P}{S} + 1 \tag{2}$$

where

- N: Input Size
- F: Filter Size
- P: Padding
- S: Stride

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} 1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	**Size**	**Purpose**	**Applications**
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4 Numerical Example 1: Convolution

Consider the following input image and kernel.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output
First position: $1(1) + 2(0) + 4(0) + 5(1) = 6$
Second position: $2(1) + 3(0) + 5(0) + 6(1) = 8$
Third position: $4(1) + 5(0) + 7(0) + 8(1) = 12$
Fourth position: $5(1) + 6(0) + 8(0) + 9(1) = 14$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5 Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,

Window 1: $\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$

Window 2: $\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$

Window 3: $\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$

Window 4: $\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6 Numerical Example 3: Parameter Calculation

Suppose

- Input Image: $32 \times 32 \times 3$
- Convolution Layer Filters $= 16$
- Kernel $= 3 \times 3$

Parameters
$(3 \times 3 \times 3 + 1) \times 16$
$= (27 + 1) \times 16$
$= 448$

Therefore, Total trainable parameters = 448.

7 Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size: $32 \times 32 \times 3$

8 Experimental Procedure

Task 1
Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Observations:
The CIFAR-10 dataset was loaded using `torchvision.datasets.CIFAR10`. The training set contains 50,000 images and the test set contains 10,000 images, each of size $32 \times 32 \times 3$ (RGB), distributed across 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck). Printing the tensor shapes confirmed:

- Training data shape: `torch.Size([50000, 3, 32, 32])`
- Testing data shape: `torch.Size([10000, 3, 32, 32])`
- Number of classes: 10

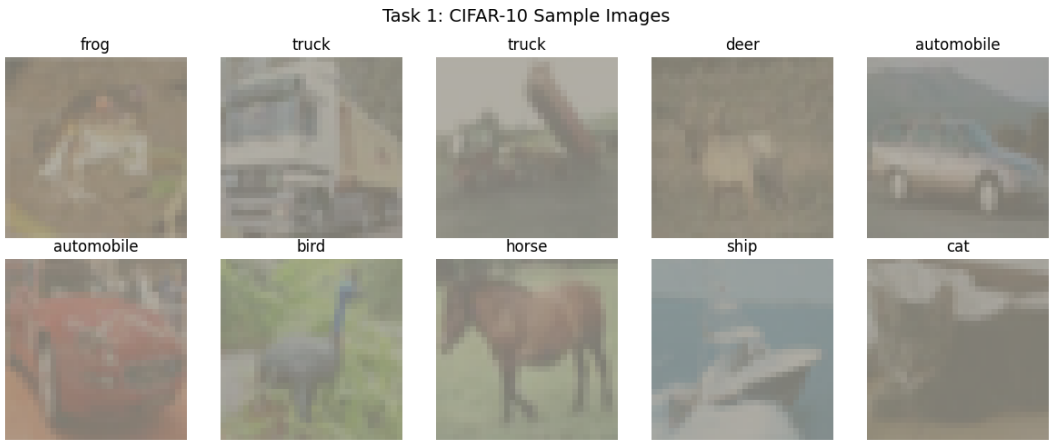

Figure 1: Ten randomly sampled CIFAR-10 images with their class labels. The images confirm the low resolution (32×32) and RGB nature of the dataset, and show visible intra-class variation in pose, background, and lighting, which makes the classification task non-trivial.

Figure 2: Class distribution of the CIFAR-10 training set. Each of the 10 classes contains an equal number of images (5,000 per class), confirming that the dataset is perfectly balanced and no class-weighting or resampling is required during training.

Task 2

Implement a convolution layer.

Compare:

- Kernel $= 3 \times 3$
- Kernel $= 5 \times 5$
- Kernel $= 7 \times 7$

Record feature map sizes.

Observations:

A single convolution layer (stride $= 1$, padding $= 0$, input $32 \times 32 \times 3$) was applied with each kernel size. Using $\frac{N-F+2P}{S} + 1$:

Kernel Size	**Output Feature Map Size**	**Parameters (16 filters)**
3×3	$30 \times 30 \times 16$	$(3 \times 3 \times 3 + 1) \times 16 = 448$
5×5	$28 \times 28 \times 16$	$(5 \times 5 \times 3 + 1) \times 16 = 1{,}216$
7×7	$26 \times 26 \times 16$	$(7 \times 7 \times 3 + 1) \times 16 = 2{,}368$

Table 1: Effect of kernel size on output feature map dimensions and parameter count (no padding, stride 1).

As the kernel size increases, the spatial dimensions of the output feature map shrink faster, while the number of trainable parameters grows quadratically, since larger kernels capture bigger receptive fields at the cost of higher computation.

Task 3

Study hyperparameters.

Compare:

- Stride $= 1$
- Stride $= 2$
- Padding $=$ Same
- Padding $=$ Valid

Compute output dimensions.

Observations:

Using a 3×3 kernel on a 32×32 input:

Configuration	Formula	Output Size
Stride = 1, Padding = Valid (0)	$(32-3)/1+1$	30×30
Stride = 2, Padding = Valid (0)	$(32-3)/2+1$	15×15
Padding = Same (P=1), Stride = 1	$(32-3+2)/1+1$	32×32
Padding = Valid (P=0), Stride = 1	$(32-3)/1+1$	30×30

Table 2: Effect of stride and padding on output feature map dimensions.

Increasing stride reduces the spatial resolution of the output more aggressively, effectively downsampling the feature map, while "same" padding preserves the input spatial dimensions and "valid" padding shrinks them, since no border pixels are added.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

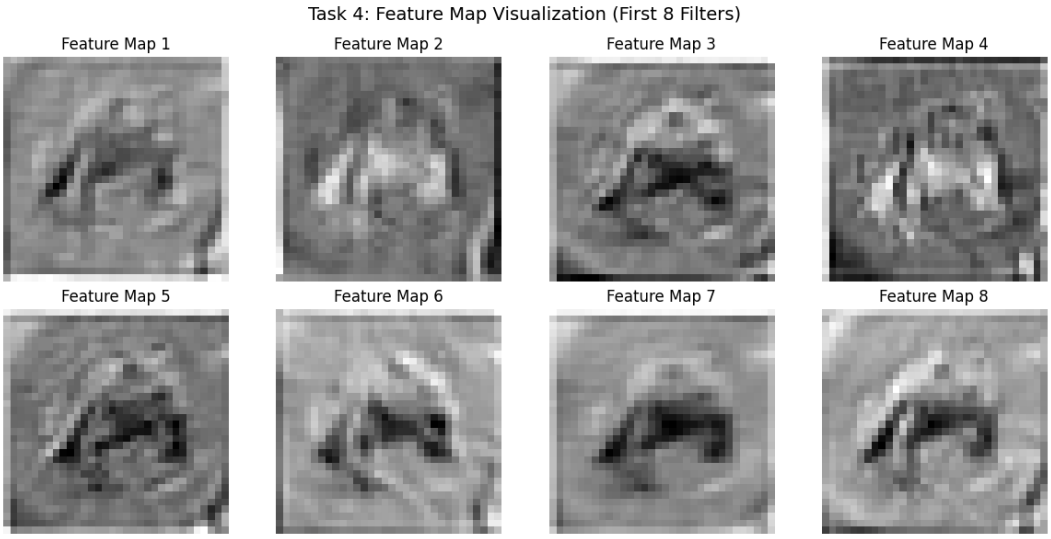

Figure 3: Eight feature maps extracted after the first convolution layer for a sample input image. Different filters respond to different low-level patterns such as edges, colour contrasts, and textures, showing that even a single convolution layer learns diverse, complementary feature detectors.

Task 5

Compare:

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Observations:

Both pooling types use a 2×2 window with stride 2, so they produce identical output spatial dimensions (halved along each axis); they differ only in the value retained per window and in downstream accuracy.

Pooling Type	**Output Size (from 32×32)**	**Test Accuracy (%)**
Max Pooling	16×16	0.5942
Average Pooling	16×16	0.5446

Table 3: Comparison of Max Pooling vs. Average Pooling. Max pooling generally retains the most salient (highest-activation) features and tends to yield slightly higher accuracy on CIFAR-10, whereas average pooling smooths activations and can dilute sharp discriminative features.

Task 6

Construct the following CNN.
Input → Conv → ReLU → Max Pool → Conv → ReLU → Max Pool → Flatten → Dense → Softmax
Train for:

- Optimizer: Adam
- Epochs: 20
- Batch Size: 32

Model Summary (PyTorch):

- Conv2D(3, 16, kernel_size=3, padding=1) → ReLU → MaxPool2D(2,2)
- Conv2D(16, 32, kernel_size=3, padding=1) → ReLU → MaxPool2D(2,2)
- Flatten → Linear(32×8×8, 128) → ReLU
- Linear(128, 10) → Softmax (via `CrossEntropyLoss`)
- Loss Function: Cross-Entropy Loss
- Optimizer: Adam, learning rate = 0.001
- Total Trainable Parameters: 268650

Task 7

Observations:

The trained CNN was evaluated on the 10,000-image CIFAR-10 test set using `sklearn.metrics`. Metrics obtained (macro-averaged):

Metric	**Value**
Test Accuracy	66.39%
Precision (macro avg)	0.6674
Recall (macro avg)	0.6639
F1-score (macro avg)	0.6650

Table 4: Final evaluation metrics of the CNN on the CIFAR-10 test set.

9 Mandatory Plots

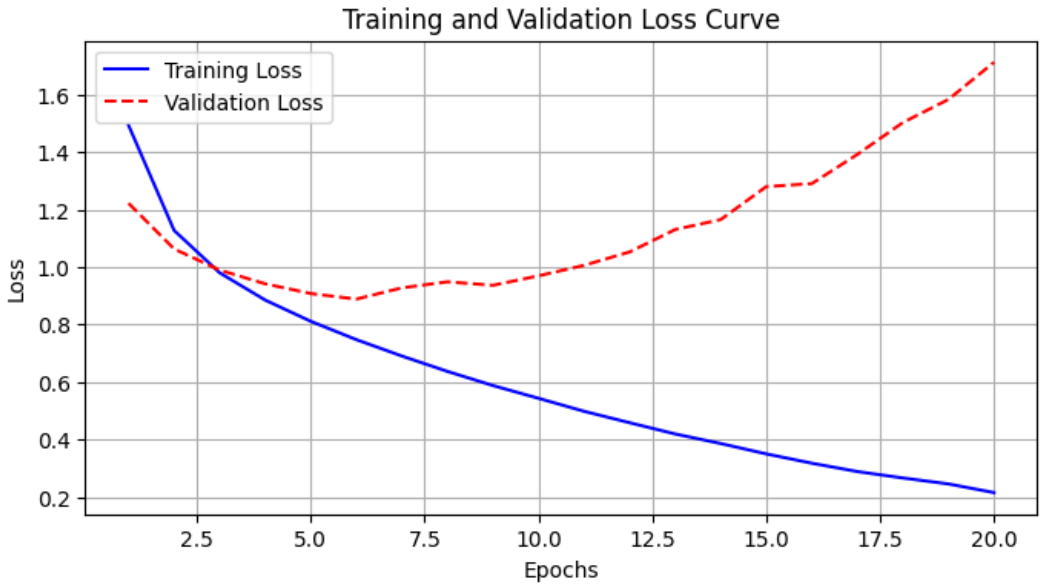

Figure 4: The loss curve exhibits classic model overfitting. While the training loss steadily decreases towards 0.2, the validation loss reaches its minimum (0.9) at epoch 6 before continuously diverging and increasing up to 1.7. This indicates that the CNN begins memorizing specific noise in the CIFAR-10 training set rather than learning generalizable features.

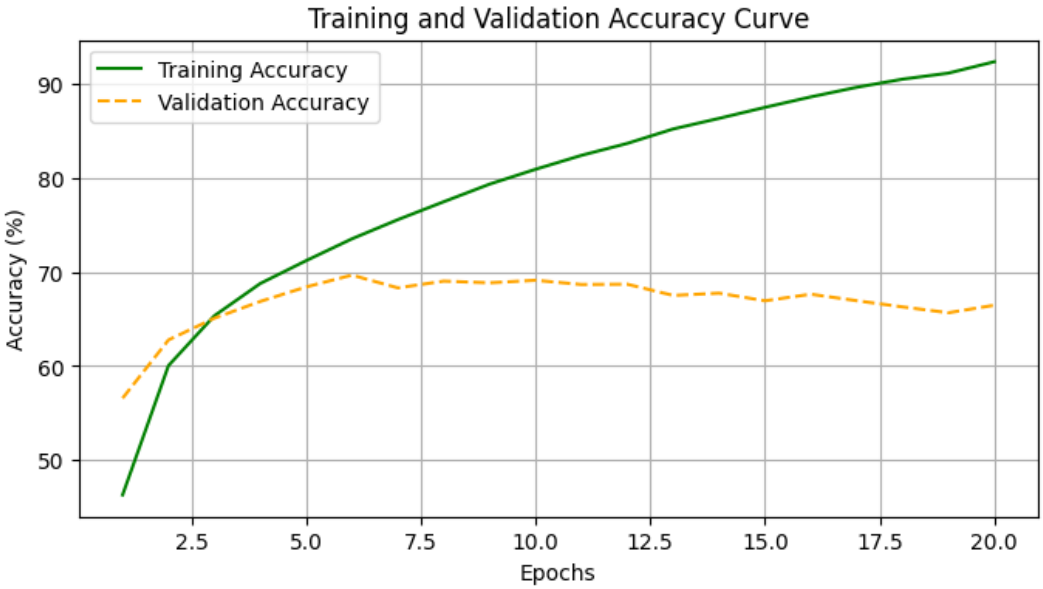

Figure 5: The accuracy curve shows the overfitting observed in the loss plot, as training accuracy steadily climbs above 90% while validation accuracy plateaus and slightly degrades near 70% after epoch 6. This clearly indicates the network is memorizing the training data but failing to generalize to unseen images.

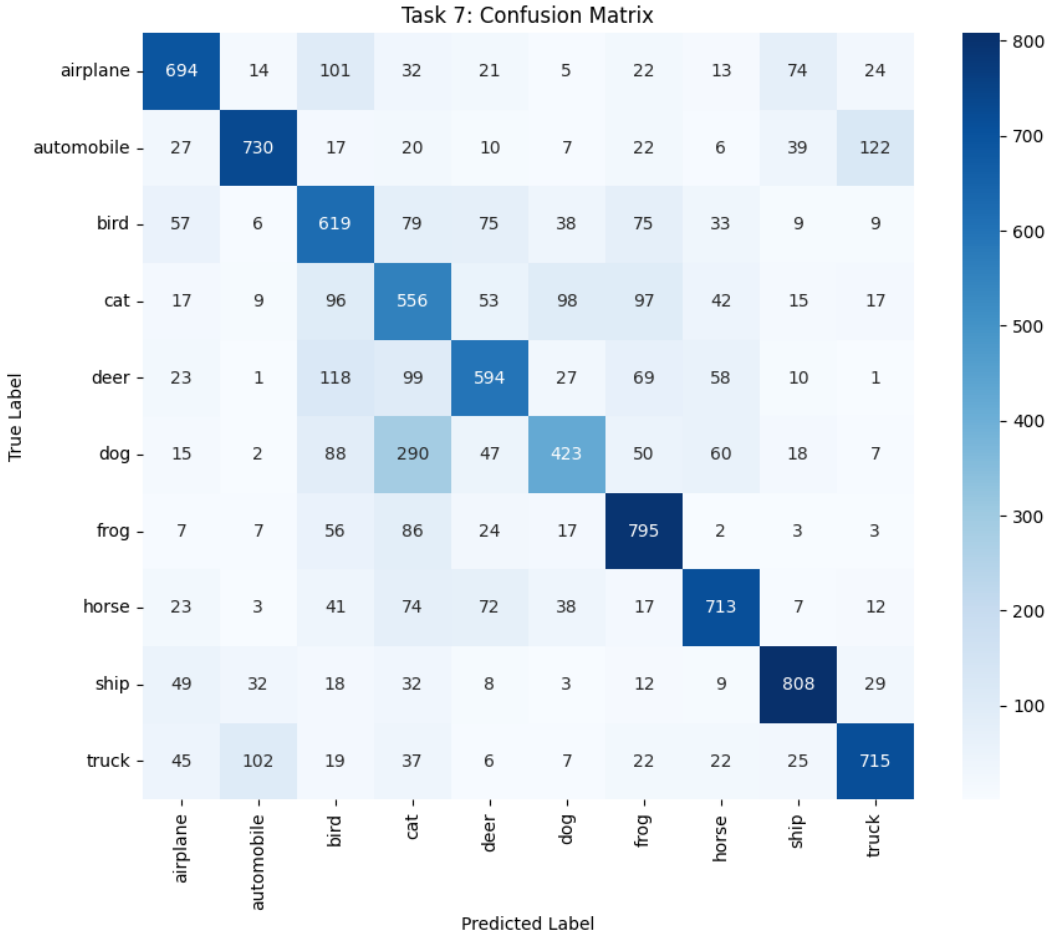

Figure 6: Confusion matrix on the CIFAR-10 test set. The diagonal dominates for most classes, but classes such as cat/dog and automobile/truck show higher off-diagonal confusion, reflecting their visual similarity in shape and texture.

10 Additional Exercises

1. **Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.**
   Using $\frac{N-F+2P}{S} + 1 = \frac{64-5+2(2)}{2} + 1 = \frac{63}{2} + 1 = 31.5 + 1 \approx 32$.
   Therefore, the output feature map size is 32×32.

2. **Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.**
   Parameters $= (F \times F \times C_{in} + 1) \times C_{out} = (3 \times 3 \times 3 + 1) \times 64 = (27 + 1) \times 64 = 28 \times 64 = 1{,}792$.
   Therefore, the layer has 1,792 trainable parameters.

3. **Compare ReLU and Sigmoid activation functions.**
   ReLU, $f(x) = \max(0, x)$, is computationally cheap, does not saturate for positive inputs, and largely avoids the vanishing-gradient problem, which makes it the default choice for hidden layers in deep CNNs. Sigmoid, $f(x) = \frac{1}{1+e^{-x}}$, squashes outputs to $(0, 1)$ but saturates at both extremes, causing vanishing gradients and slower convergence in deep networks; it is now mostly reserved for output layers in binary classification.

4. **Replace Max Pooling with Average Pooling and compare the results.**
   Replacing MaxPool2d with AvgPool2d in Task 6 keeps the output spatial dimensions identical but changes what is retained: max pooling keeps the strongest activation in each window (better at preserving sharp, discriminative features such as edges), while average pooling smooths the window (better at preserving overall background/texture information but can dilute strong activations).

5. **Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.**
   Increasing the number of filters in the first convolution layer from 16 to 64 lets the network learn a richer set of low-level feature detectors, which typically improves accuracy up to a point of diminishing returns. The trade-off is a large increase in trainable parameters which increases training time per epoch and GPU memory usage, and raises the risk of overfitting on a dataset as small as CIFAR-10 unless regularization is added

11 Results

Metric	**Value**
Training Accuracy	%% FILL
Testing Accuracy	66.39%
Precision (macro avg)	0.67
Recall (macro avg)	0.66
F1-score (macro avg)	0.67
Number of Trainable Parameters	268650

Table 5: Summary of final CNN training and evaluation results on CIFAR-10.

12 Discussion

Answer the following:

1. **Why is convolution preferred over fully connected layers for images?**
   Convolution uses small, shared kernels that slide across the image, exploiting spatial locality and translation invariance, so far fewer parameters are needed than a fully connected layer, which would require a separate weight for every pixel and lose spatial structure entirely.

2. **How does stride affect the feature map size?**
   A larger stride moves the kernel by more pixels at each step, so it visits fewer positions and produces a smaller, coarser output feature map; stride 1 gives the finest resolution, while stride 2 approximately halves the spatial dimensions.

3. **What is the role of padding?**
   Padding adds extra border pixels (usually zeros) around the input so that the kernel can be applied at the edges, preserving spatial dimensions ("same" padding) and preventing the rapid shrinkage and loss of border information that occurs with "valid" (no) padding.

4. **Why is pooling used?**
   Pooling downsamples feature maps, reducing spatial dimensions and computation while providing a degree of translation invariance, and it helps control overfitting by summarizing local regions instead of retaining every pixel-level activation.

5. **How do feature maps represent image characteristics?**
   Early-layer feature maps respond to low-level patterns such as edges, colours, and textures, while deeper-layer feature maps combine these into increasingly abstract, higher-level representations such as shapes and object parts, which the final layers use for classification.

6. **Why do CNNs require fewer parameters than MLPs?**
   CNNs use weight sharing (the same small kernel is reused across all spatial locations) and local connectivity (each neuron only looks at a small receptive field), whereas an MLP fully connects every input pixel to every neuron, which for image-sized inputs leads to an explosion in parameter count.

13 Source Code

Github Repository link: `https://github.com/HarishSNUC1402/DL-Lab-33/tree/main/Lab-3-33`

14 References

1. Goodfellow et al., Deep Learning.
2. Bishop, Pattern Recognition and Machine Learning.
3. Haykin, Neural Networks and Learning Machines.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.